

Министерство образования Республики Беларусь

Учреждение образования

“БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ”

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1

по курсу “Модели решения задач в
интеллектуальных системах”

Вариант 13

Выполнил:
студент гр. 221702

Потоцкий Д.А.

Проверил:

Ивашенко В. П.

Минск
2025

Тема: Рециркуляционная сеть.

Цель: Ознакомиться, проанализировать и получить навыки реализации модели линейной рециркуляционной сети для задачи сжатия графической информации.

Задание: Реализовать модель линейной рециркуляционной сети с постоянным коэффициентом обучения с ненормированными весами.

Постановка задачи:

В данной лабораторной работе изучается принцип функционирования линейной рециркуляционной нейронной сети (автоэнкодера), применяемой для сжатия графической информации. Необходимо реализовать модель сети с постоянным коэффициентом обучения, провести обучение на различных изображениях, построить необходимые графики для анализа процесса обучения и ошибки обученной модели, а также ответить на контрольные вопросы по теме.

Описание реализации

Программа принимает на вход изображение в формате BMP для обучения модели нейронной сети, предназначенной для сжатия графической информации. В качестве выходных данных программа формирует восстановленное изображение в том же формате.

Реализация выполнена на языке программирования Java (версия 21) с использованием следующих библиотек и подходов:

- Picocli — для удобной и мощной обработки аргументов командной строки.
- Стандартные библиотеки Java — BufferedImage и ImageIO для чтения, записи и манипуляции с изображениями.
- Виртуальные потоки (Project Loom) — применяются для распараллеливания матричных вычислений с целью повышения производительности.
- Сторонние библиотеки отсутствуют, все вычисления инкапсулированы в специально разработанном для этого классе Matrix.

Были использованы следующие структуры данных:

- Класс Matrix — для хранения данных изображения, весов нейронов и выполнения матричных операций (умножение, вычитание, транспонирование).
- Класс CliArguments — для инкапсуляции и парсинга параметров командной строки.
- Записи (record) — для создания неизменяемых объектов-контейнеров, например, для хранения блоков изображения и его исходных размеров.

Структура Java-проекта выглядит следующим образом:

- ImageCompressor.java — основной класс, управляющий всем процессом: чтение аргументов, подготовка данных, запуск обучения сети и сохранение результата.

- `CliArguments.java` — класс для описания и парсинга аргументов командной строки с помощью аннотаций `Picocli`.
- `image/ImageUtils.java` — модуль для предобработки и восстановления изображений. Реализует разделение изображения на блоки, их нормализацию в диапазон $[-1, 1]$ и обратную сборку.
- `network/NeuralNetwork.java` — модуль реализации линейного автоэнкодера. Обеспечивает прямое и обратное распространение ошибки, а также реализует цикл обучения.
- `network/Matrix.java` — вспомогательный класс для всех матричных операций.

Работа программы осуществляется по следующему алгоритму:

1. Получение аргументов: При запуске программы осуществляется парсинг аргументов командной строки с помощью библиотеки `Picocli`. Вводятся параметры: размеры блоков изображения m и n ; количество нейронов скрытого слоя; максимальное количество эпох `maxEpochs`; максимальное допустимое значение ошибки `maxError`; коэффициент обучения lr ; пути к входному и выходному файлам.
2. Чтение и подготовка изображения: Производится чтение входного изображения в формате BMP. Пиксели преобразуются в массив числовых значений, где каждый канал (R, G, B) нормализуется в диапазон $[-1.0, 1.0]$.
3. Разделение на блоки: Изображение дополняется до размеров, кратных размеру блока ($m \times n$), после чего делится на непересекающиеся прямоугольные области. Каждая область преобразуется в одномерный вектор входных данных для нейронной сети.
4. Определение параметров сети: На основе размера блока вычисляется размер входного и выходного слоёв ($N_{in} = N_{out} = 3 * m * n$). Размер скрытого слоя (N_{hidden}) по умолчанию равен $N_{in}/2$.
5. Инициализация сети: Создается экземпляр `NeuralNetwork`. Весовые матрицы инициализируются псевдослучайными значениями, равномерно распределёнными в диапазоне $[-1.0, 1.0]$.
6. Прямой проход: Для каждого отдельного блока изображения выполняется прямое распространение сигнала: входной вектор умножается на матрицу весов первого слоя ($W1$), а результат — на матрицу весов второго слоя ($W2$), формируя восстановленный блок на выходе.
7. Вычисление ошибки и обратный проход: После прямого прохода вычисляется вектор ошибки как разность между исходным и восстановленным блоками. На основе этой ошибки производится немедленная коррекция весовых матриц $W1$ и $W2$ методом обратного распространения ошибки (стохастический градиентный спуск).
8. Итерационное повторение: Шаги 6 и 7 повторяются последовательно для

всех блоков изображения. Проход по всем блокам составляет одну эпоху обучения.

9. Проверка условия остановки: После завершения каждой эпохи вычисляется суммарная ошибка для всего набора данных. Если она ниже порога $\maxError$ или достигнуто максимальное число эпох $\maxEpochs$, процесс обучения завершается. В противном случае начинается новая эпоха.
10. Восстановление и сохранение: По завершении обучения все блоки исходного изображения пропускаются через обученную сеть. Полученные восстановленные блоки денормализуются и собираются в итоговое изображение, которое сохраняется в выходной файл.

Описание модели:

Программа реализует модель линейного автоэнкодера, который является развёрткой рециркуляционной сети и состоит из трёх слоёв: входного, скрытого (кодировющего) и выходного (декодировющего). Прямое распространение информации выполняется путем последовательного матричного умножения входного вектора на матрицу весов кодировющего слоя ($W1$), а затем полученного сжатого вектора — на матрицу весов декодировющего слоя ($W2$). Процесс обучения реализует метод обратного распространения ошибки. Ошибка вычисляется как разность между исходными значениями пикселей и восстановленными. Эта ошибка используется для итеративной коррекции весовых матриц $W1$ и $W2$ с применением метода градиентного спуска с постоянным коэффициентом обучения ($learningRate$). Скрытый слой, таким образом, обучается эффективно кодировать информацию, а выходной — декодировать ее с минимальными потерями, что обеспечивает функциональность сети для сжатия и последующей реконструкции изображений.

Условные обозначения:

m — ширина прямоугольника;
 n — высота прямоугольника;
 i — число итераций;
 E — суммарная ошибка.
 e — максимальная допустимая ошибка;
 lr — коэффициент обучения;
 Z — коэффициент сжатия;

Тестирование

На вход линейной рециркуляционной сети подается изображение(рис.1):



Рис. 1. Входное изображение

А также заданы параметры сети:

$m = n = 8$, $e = 5000$, $lr = 0.0001$, количество нейронов на скрытом слое: 20
Коэффициент сжатия: 1.1463. Обучение сети прошло за 75 секунд.

Тестирование модели прошло успешно. На выходе было получено изображение (рис. 2), которое визуально соответствует исходному и сохраняет основные детали и структуру. Незначительные отличия связаны с потерями, возникающими при сжатии, что является ожидаемым результатом для линейной модели. Это подтверждает корректность реализации алгоритма, устойчивость обучения и работоспособность разработанной рециркуляционной сети.



Рис. 2. Выходное изображение

Графики

Параметры для графика зависимости среднего количества итераций от коэффициента сжатия:

Используется детерминированный генератор псевдослучайных чисел

Размер изображения $512 * 512$

$m = n = 8$

$e = 5000$

$lr = 0.0001$

График зависимости среднего количества итераций от коэффициента сжатия(рис. 3)

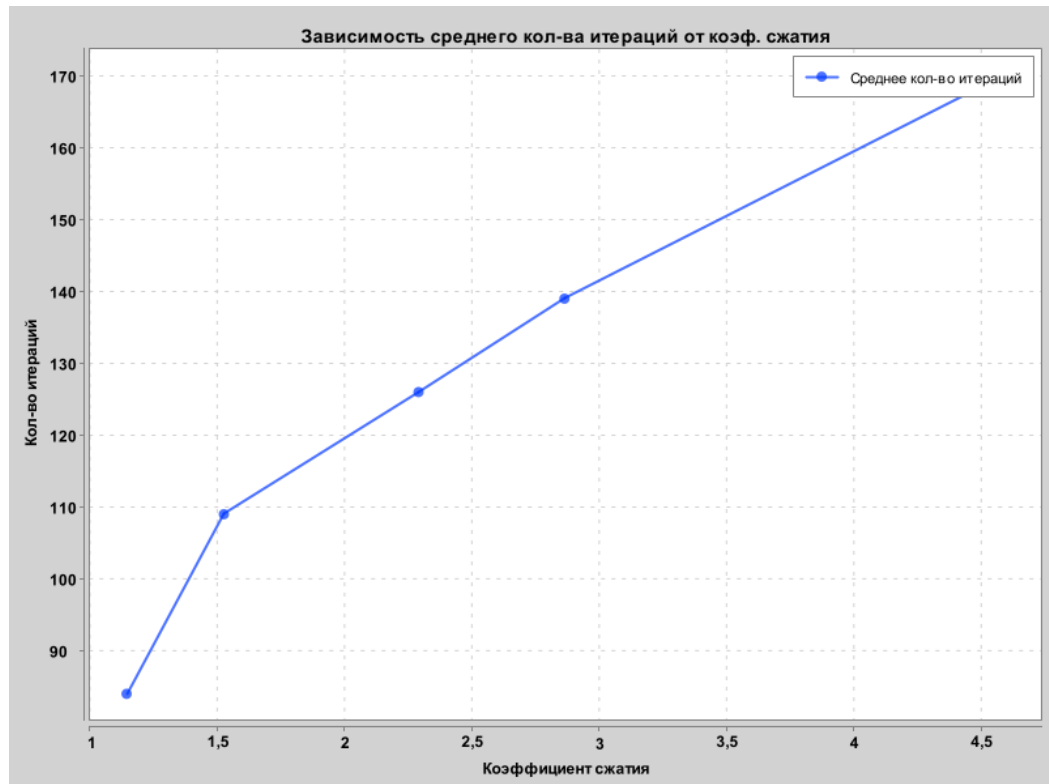


Рис. 3. График зависимости среднего количества итераций от коэффициента сжатия

График зависимости количества итераций от максимально допустимой ошибки (рис. 4)

Параметры:

Размер изображения 512 * 512

$m = n = 8$

Кол-во нейронов на скрытом слое: 20

Коэффициент сжатия: 1,1463

$lr = 0.0001$

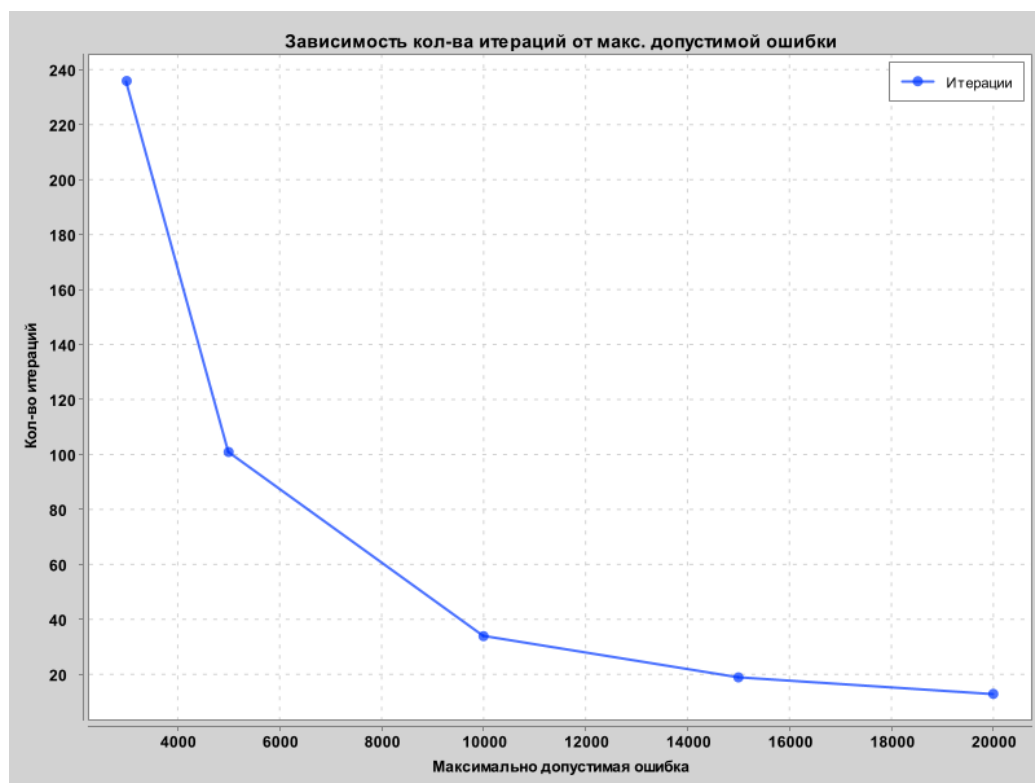


Рис. 4. График зависимости количества итераций от максимально допустимой ошибки

График зависимости количества итераций от коэффициента обучения(рис. 5)

Параметры:

Размер изображения 512 * 512

$m = n = 8$

Кол-во нейронов на скрытом слое: 20

Коэффициент сжатия: 1,1463

$\epsilon = 5000$

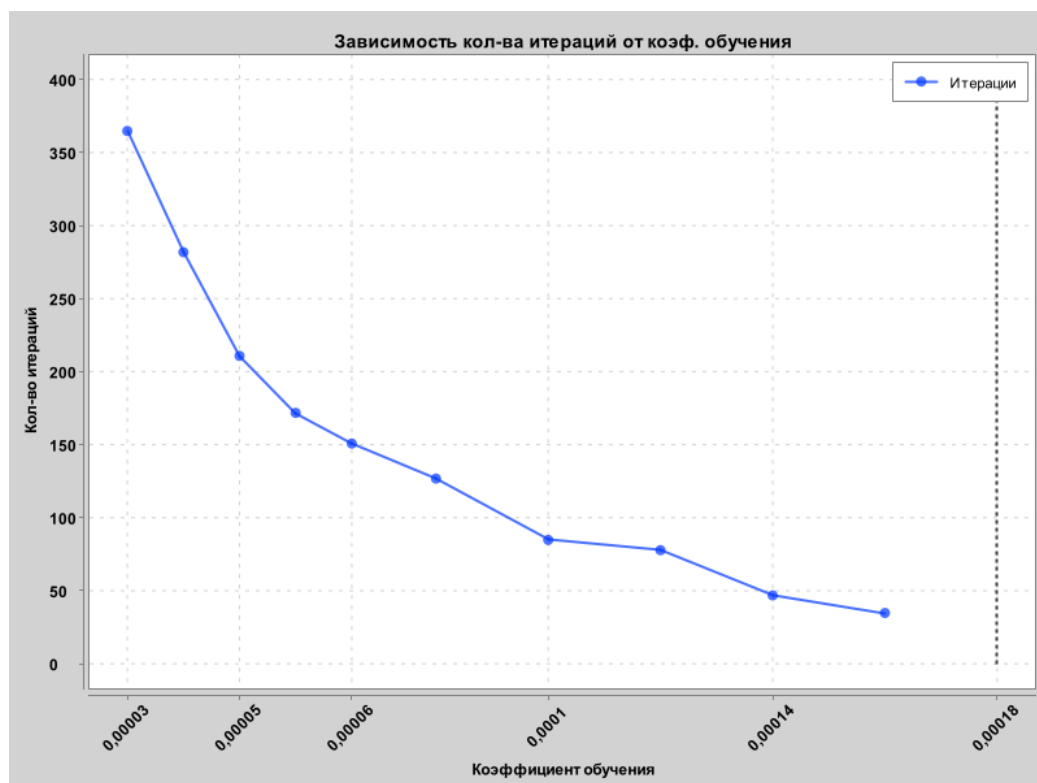


Рис. 5. График зависимости количества итераций от коэффициента обучения

Зависимость количества итераций от максимально допустимой ошибки на разных изображениях

Параметры:

Размер изображения 512 * 512

$m = n = 8$

Кол-во нейронов на скрытом слое: 20

Коэффициент сжатия: 1,1463

$lr = 0.0001$

Первое изображение подаваемое на вход(рис. 6)



Рис. 6. Первое изображение подаваемое на вход

Второе изображение подаваемое на вход (рис. 7)



Рис. 7. Второе изображение подаваемое на вход

Третье изображение подаваемое на вход (рис. 8)

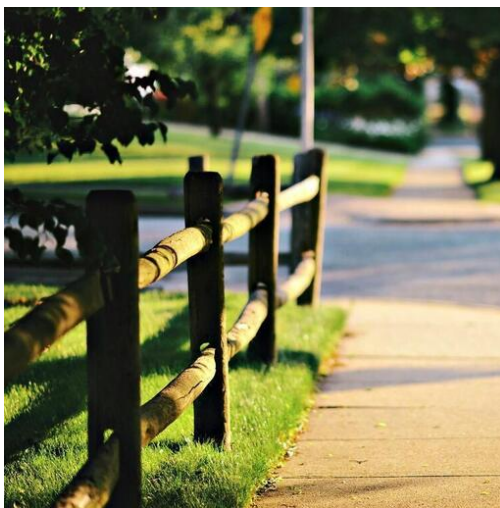


Рис. 8. Третье изображение подаваемое на вход

График зависимости количества итераций от максимально допустимой ошибки(рис. 9)

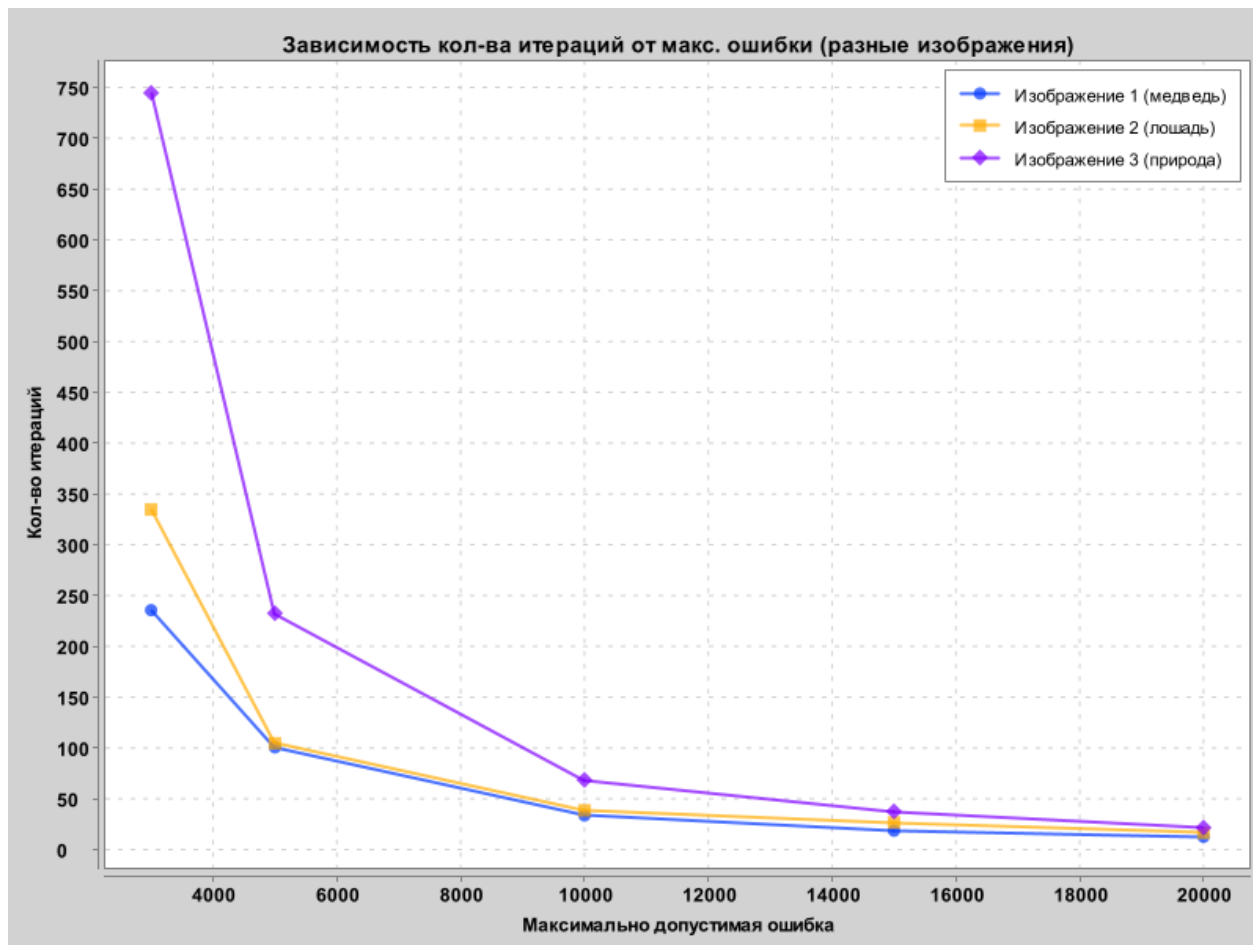


Рис. 9. График зависимости количества итераций от максимально допустимой ошибки

Ответы на контрольные вопросы

- Какова зависимость количества итераций обучения до достижения заданного значения ошибки от величины коэффициента обучения(рис.5)?

Наблюдение	Кол-во итераций	Коэффициент обучения (lr)
1	365	0.000025
2	282	0.000035
3	211	0.000045
4	172	0.000055
5	151	0.000065
6	127	0.00008
7	85	0.0001

Анализ данных, полученных в ходе эксперимента, демонстрирует выраженную

обратную зависимость между коэффициентом обучения (lr) и количеством итераций, необходимых для достижения целевой ошибки. С увеличением значения коэффициента обучения число итераций, требуемых для сходимости сети, значительно уменьшается. При минимальном lr (0.000025) потребовалось максимальное количество итераций (365), так как сеть делала очень маленькие шаги при коррекции весов. В то время как при увеличении lr до 0.0001 число итераций упало до 85. Это подтверждает, что правильно подобранный коэффициент обучения критически важен для ускорения сходимости. Однако стоит помнить, что чрезмерно высокий коэффициент может привести к нестабильности обучения и перепрыгиванию через оптимальное решение.

2. Какова зависимость количества итераций обучения до достижения заданного значения ошибки от величины коэффициента сжатия(рис.3)?

Наблюдение	Среднее кол-во итераций	Коэффициент сжатия (Z)
1	109	4.5849
2	71	3.0566
3	61	2.2925
4	55	1.8340
5	53	1.5283

Исследование зависимости от коэффициента сжатия (Z) показывает, что более сильное сжатие (меньшее значение Z) способствует более быстрой сходимости сети. Анализ показывает, что с уменьшением коэффициента сжатия Z среднее количество итераций также стабильно снижается. Например, при слабом сжатии ($Z \approx 4.6$) потребовалось 170 итераций, а при более сильном ($Z \approx 1.1$) — всего 84. Это говорит о том, что модель с более узким "бутылочным горлышком", то есть меньшим скрытым слоем, обучается быстрее. Это связано с тем, что сети приходится изучать более компактное и обобщенное представление данных, что упрощает задачу оптимизации.

3. Какова зависимость количества итераций обучения до достижения заданного значения ошибки от вида сжимаемого изображения(рис. 9)?

Наблюдение	Изображение 1 (медведь)	Изображение 2 (лошадь)	Изображение 3 (природа)	Ошибка (e)
1	236	335	745	3000
2	101	105	233	5000
3	34	39	69	10000
4	19	27	38	15000
5	13	17	22	20000

Результаты эксперимента наглядно демонстрируют, что количество итераций

обучения существенно зависит от структуры и сложности содержимого изображения. Наиболее сложным для сети оказалось изображение природа, которое требовало наибольшего числа итераций на всех уровнях ошибки (например, 745 итераций при $\epsilon=3000$). Наименьшее количество итераций потребовалось для изображения медведь (236 итераций при $\epsilon=3000$). Это указывает на то, что изображения с более сложной текстурой, большим количеством мелких деталей и разнообразной цветовой палитрой требуют от сети значительно больше времени на обучение по сравнению с изображениями с более однородными областями. При этом для всех трех изображений наблюдается общая тенденция: с ростом допустимой ошибки количество итераций закономерно падает.

4. Какова зависимость количества итераций обучения до достижения заданного значения ошибки от его величины(рис.4)?

Наблюдение	Кол-во итераций	Ошибка (ϵ)
1	236	3000
2	101	5000
3	34	10000
4	19	15000
5	13	20000

Анализ данных показывает строгую обратную зависимость между количеством итераций обучения и величиной максимально допустимой ошибки (ϵ). Чем выше порог допустимой ошибки, тем меньше итераций требуется для завершения обучения. При строгом пороге ($\epsilon=3000$) потребовалось 236 итераций для его достижения. Когда порог был значительно ослаблен ($\epsilon=20000$), сети понадобилось всего 13 итераций. Это логичный результат, поскольку более высокое значение ϵ является менее строгим критерием останова, что позволяет процессу обучения завершиться раньше, как только модель достигает заданного, менее точного, уровня аппроксимации исходных данных.

Вывод: В ходе лабораторной работы была реализована и исследована модель линейной рециркуляционной нейронной сети с постоянным коэффициентом обучения для задачи сжатия изображений. Было проведено экспериментальное исследование влияния ключевых гиперпараметров, таких как коэффициент обучения, коэффициент сжатия и максимально допустимая ошибка, на эффективность и скорость обучения модели. Обучение и тестирование проводились на наборе из трех различных изображений.

В результате проведенных экспериментов были установлены следующие ключевые зависимости:

- **Коэффициент обучения (lr):** Установлена выраженная обратная зависимость между скоростью обучения и количеством итераций. Повышение коэффициента lr в исследуемом диапазоне значительно сокращает количество итераций, необходимых для достижения целевой ошибки, тем самым ускоряя сходимость сети.
- **Коэффициент сжатия (Z):** Увеличение степени сжатия (то есть уменьшение коэффициента Z за счет уменьшения размера скрытого слоя) приводит к сокращению среднего времени обучения. Это указывает на то, что модель с более компактным скрытым слоем быстрее обучается обобщенным, низкоуровневым признакам изображения.
- **Сложность изображения:** Сложность и структура исходного изображения являются определяющими факторами для скорости обучения. Эксперименты показали, что изображения с высокой детализацией и сложной текстурой требуют значительно большего числа итераций по сравнению с более однородными изображениями для достижения того же уровня ошибки.
- **Допустимая ошибка (ϵ):** Подтверждена очевидная обратная зависимость: количество итераций обратно пропорционально величине допустимой ошибки. Более высокие (менее строгие) значения ϵ позволяют достичь формальной сходимости быстрее, но за счет потенциального снижения качества восстановленного изображения.

Таким образом, работа подтверждает, что эффективность обучения линейного автоэнкодера напрямую зависит от сбалансированного подбора гиперпараметров. Правильный выбор коэффициента обучения, степени сжатия и целевой ошибки позволяет найти оптимальное соотношение между скоростью сходимости сети и качеством итогового сжатого изображения, адаптируясь под конкретные характеристики входных данных.

Список использованных источников

1. Формальные модели обработки информации и параллельные модели решения задач. Практикум: учебно-методическое пособие / В.П.Ивашенко. – Минск: БГУИР, 2020.
2. Методические указания к лабораторной работе.